

CO2 AT1

Q2. LOCAL VS. GLOBAL HISTOGRAM STRETCHING

Objective

To compare Global Histogram Equalization and Contrast Limited Adaptive Histogram Equalization (CLAHE) and analyse their performance on an image with non-uniform illumination.

Tool Used

Python, OpenCV, NumPy, Matplotlib and Visual Studio Code.

Input Image

The standard Lena image is used as the input image. Non-uniform illumination is artificially introduced into the image by creating a gradual intensity variation from one side of the image to the other.



Figure 1: Original Input Image

Procedure

1. Read the input image using OpenCV.
2. Convert the colour image into grayscale.
3. Introduce non-uniform illumination into the image.
4. Apply Global Histogram Equalization using `cv2.equalizeHist()`.
5. Apply CLAHE with a clip limit of **4.0** and grid size of **8 × 8**.
6. Generate the intensity histograms.

7. Compare the outputs obtained from Global Histogram Equalization and CLAHE.

Program

```
import cv2
import matplotlib.pyplot as plt
import os

# Create outputs folder if it doesn't exist
os.makedirs("outputs", exist_ok=True)

# Read image
image = cv2.imread("images/histogram.jpg")
if image is None:
    print("Error: histogram.jpg not found!")
    exit()

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Global Histogram Equalization
global_eq = cv2.equalizeHist(gray)

# CLAHE
clahe = cv2.createCLAHE(clipLimit=4.0, tileGridSize=(8,8))
clahe_img = clahe.apply(gray)

# Save images
cv2.imwrite("outputs/q2_original.jpg", gray)
cv2.imwrite("outputs/q2_global.jpg", global_eq)
cv2.imwrite("outputs/q2_clahe.jpg", clahe_img)

# Display
cv2.imshow("Original", gray)
cv2.imshow("Global Histogram Equalization", global_eq)
cv2.imshow("CLAHE", clahe_img)

# Histogram
plt.figure(figsize=(8,5))
plt.hist(gray.ravel(),256,[0,256],label="Original")
plt.hist(global_eq.ravel(),256,[0,256],label="Global")
plt.hist(clahe_img.ravel(),256,[0,256],label="CLAHE")
plt.title("Histogram Comparison")
```

```
plt.xlabel("Pixel Intensity")  
plt.ylabel("Frequency")  
plt.legend()  
plt.savefig("outputs/q2_histogram.jpg")  
plt.show()  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

Output :



Figure 2: Image with Non-Uniform Illumination



Figure 3: Global Histogram Equalization Output



Figure 4: CLAHE Output



Figure 5: Histogram Comparison

Analysis

Global Histogram Equalization considers the intensity distribution of the entire image and applies a single transformation to all pixels. Therefore, when the image contains regions with different illumination levels, some regions may become over-enhanced while details in other regions may be lost.

CLAHE performs histogram equalization on small local regions or tiles. The clip limit restricts excessive contrast amplification. With a clip limit of 4.0 and an 8×8 grid, CLAHE provides better local contrast and preserves details in both darker and brighter regions.

Hence, CLAHE performs better than Global Histogram Equalization for images affected by non-uniform illumination.

Result

Global Histogram Equalization and CLAHE were successfully implemented and compared. CLAHE produced better local contrast and preserved more image details under non-uniform illumination.

Q5. NON-LINEAR MEDIAN FILTER DESTRUCTIVE TESTING

Objective

To introduce 15% Salt and Pepper noise into an image and compare the performance of a 3×3 Gaussian filter and a 3×3 Median filter.

Tool Used

Python, OpenCV, NumPy and Visual Studio Code.

Input Image :



Code :

```
import cv2
import numpy as np
import os
os.makedirs("outputs", exist_ok=True)
image = cv2.imread("images/lena.jpg")
if image is None:
    print("Error: lena.jpg not found!")
    exit()
noisy = image.copy()
probability = 0.15
random = np.random.rand(image.shape[0], image.shape[1])
```

```
noisy[random < probability/2] = 0
noisy[random > 1-probability/2] = 255
gaussian = cv2.GaussianBlur(noisy,(3,3),0)
median = cv2.medianBlur(noisy,3)
cv2.imwrite("outputs/q5_noise.jpg", noisy)
cv2.imwrite("outputs/q5_gaussian.jpg", gaussian)
cv2.imwrite("outputs/q5_median.jpg", median)
cv2.imshow("Original", image)
cv2.imshow("Salt & Pepper Noise", noisy)
cv2.imshow("Gaussian Filter", gaussian)
cv2.imshow("Median Filter", median)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :



Figure 2: Output after 3×3 Gaussian Filtering



Figure 3: Output after 3×3 Median Filtering



Figure 4: Image with 15% Salt and Pepper Noise

Analysis

Salt and Pepper noise introduces random black and white pixels into an image, which degrades its visual quality.

The **Gaussian Filter** removes noise by averaging neighbouring pixel values. Although it reduces the effect of the noise, it also blurs image edges and fine details because it smooths the entire image uniformly.

The **Median Filter** is a non-linear filtering technique that replaces each pixel with the median value of its neighbouring pixels. Since the median operation effectively removes isolated black and white pixels without averaging, it preserves edges while eliminating Salt and Pepper noise.

From the obtained results, the Median Filter produces a cleaner image with sharper edges compared to the Gaussian Filter. Therefore, the Median Filter is more suitable for removing Salt and Pepper noise.

Result

The Gaussian Filter and Median Filter were successfully implemented and compared. The **Median Filter** removed Salt and Pepper noise more effectively while preserving important image details and edges, making it the better filtering technique for impulse noise removal.

Q6. 2D FAST FOURIER TRANSFORM (FFT) PHASE FILTERING

Objective

To perform **2D Fast Fourier Transform (FFT)** on an image, remove the high-frequency components using a low-pass filter, reconstruct the image using the Inverse FFT, and analyse the effect of frequency-domain filtering on image quality.

Tool Used

Python, OpenCV, NumPy and Visual Studio Code.

Input Image :



Figure 1: Original Input Image

Program :

```
import cv2
import numpy as np
import os

os.makedirs("outputs", exist_ok=True)

image = cv2.imread("images/cameraman.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: cameraman.jpg not found!")
```

```

    exit()
fft = np.fft.fft2(image)
fft_shift = np.fft.fftshift(fft)
magnitude = np.log(np.abs(fft_shift) + 1)
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2
mask = np.zeros((rows, cols), np.uint8)
cv2.circle(mask, (ccol, crow), 40, 1, -1)
filtered = fft_shift * mask
inverse = np.fft.ifft2(np.fft.ifftshift(filtered))
inverse = np.abs(inverse)
inverse = cv2.normalize(inverse, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)
spectrum = cv2.normalize(magnitude, None, 0, 255,
                        cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("outputs/q6_fft_spectrum.jpg", spectrum)
cv2.imwrite("outputs/q6_filtered.jpg", inverse)
cv2.imshow("Original", image)
cv2.imshow("FFT Spectrum", spectrum)
cv2.imshow("Filtered Image", inverse)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Output :

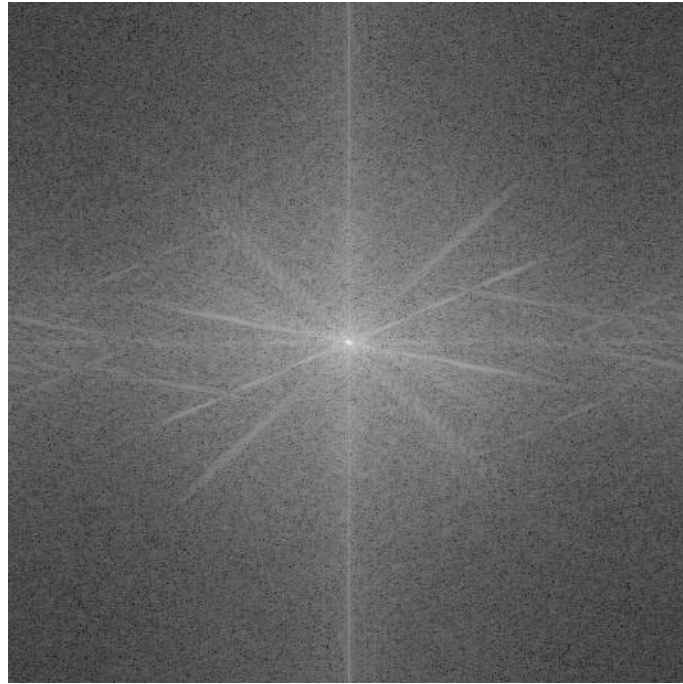


Figure 2: FFT Magnitude Spectrum



Figure 3: Reconstructed Image after High-Frequency Removal

Analysis

The Fast Fourier Transform (FFT) converts an image from the spatial domain into the frequency domain, where different frequency components represent different image characteristics.

Low-frequency components contain the overall brightness and smooth regions of the image, whereas high-frequency components represent edges, textures and fine details.

A circular low-pass mask is applied to retain only the low-frequency components while removing the high-frequency information. After applying the Inverse FFT, the reconstructed image appears smoother because most of the sharp edges and fine details have been removed.

This experiment demonstrates that frequency-domain filtering can effectively smooth an image by eliminating high-frequency components while preserving the overall structure of the image.

Result

The 2D Fast Fourier Transform (FFT) and frequency-domain low-pass filtering were successfully implemented. The reconstructed image became smoother after removing the high-frequency components, demonstrating the effectiveness of FFT-based image filtering.

Q8. PYTHON OPENCV EDGE ENHANCEMENT SANDBOX

Objective

To perform edge detection using **Sobel X**, **Sobel Y**, **Gradient Magnitude**, and **Laplacian** operators, and compare the effect of 3×3 and 5×5 Sobel kernels on image edge enhancement.

Tool Used

Python, OpenCV, NumPy and Visual Studio Code.

Input Image :



Figure 1: Original Input Image

Procedure

1. Read the input image in grayscale using OpenCV.
2. Apply the Sobel X operator using a 3×3 kernel.
3. Apply the Sobel Y operator using a 3×3 kernel.
4. Calculate the Gradient Magnitude using the Sobel X and Sobel Y outputs.
5. Apply the Laplacian operator for edge detection.
6. Apply the Sobel X operator again using a 5×5 kernel.

7. Display all processed images.
8. Save the output images for comparison.
9. Compare the edge enhancement results obtained using different operators.

Program :

```
import cv2
import numpy as np
import os
os.makedirs("outputs", exist_ok=True)
image = cv2.imread("images/bricks.jpg", cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: bricks.jpg not found!")
    exit()
sobelx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
gradient = cv2.magnitude(
    sobelx.astype(np.float32),
    sobely.astype(np.float32)
)
laplacian = cv2.Laplacian(image, cv2.CV_64F)
sobel5 = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=5)
sobelx = cv2.convertScaleAbs(sobelx)
sobely = cv2.convertScaleAbs(sobely)
gradient = cv2.convertScaleAbs(gradient)
laplacian = cv2.convertScaleAbs(laplacian)
sobel5 = cv2.convertScaleAbs(sobel5)
cv2.imwrite("outputs/q8_sobelx.jpg", sobelx)
cv2.imwrite("outputs/q8_sobely.jpg", sobely)
cv2.imwrite("outputs/q8_gradient.jpg", gradient)
cv2.imwrite("outputs/q8_laplacian.jpg", laplacian)
```

```
cv2.imwrite("outputs/q8_sobel5.jpg", sobel5)
cv2.imshow("Original", image)
cv2.imshow("Sobel X", sobelx)
cv2.imshow("Sobel Y", sobely)
cv2.imshow("Gradient Magnitude", gradient)
cv2.imshow("Laplacian", laplacian)
cv2.imshow("Sobel 5x5", sobel5)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output :

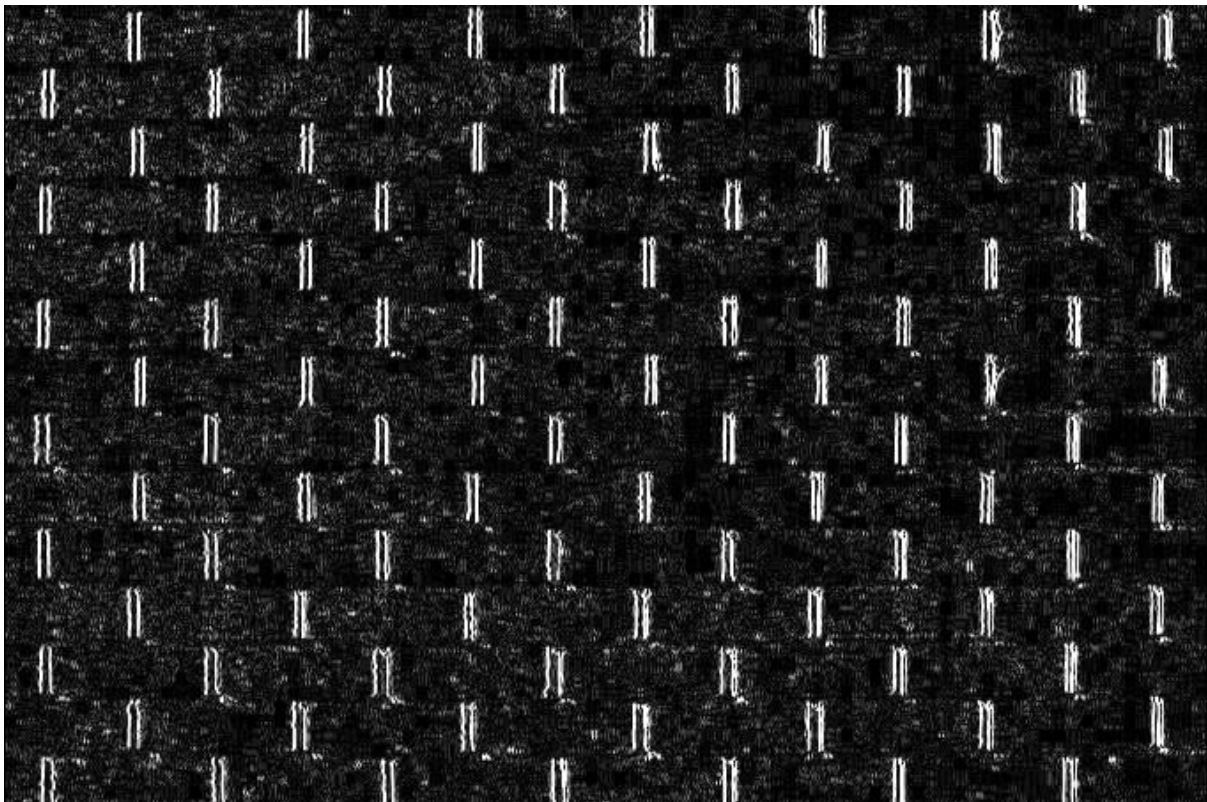


Figure 2: Sobel X Edge Detection (3×3 Kernel)

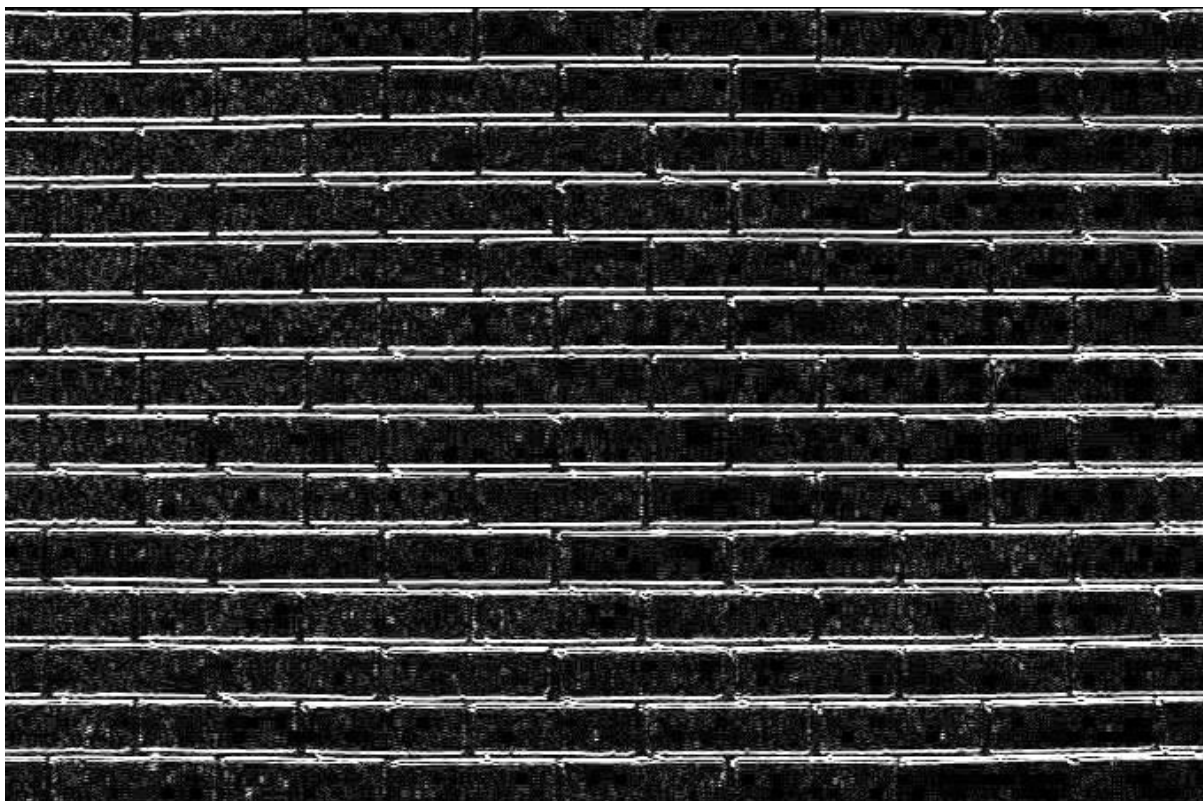


Figure 3: Sobel Y Edge Detection (3×3 Kernel)

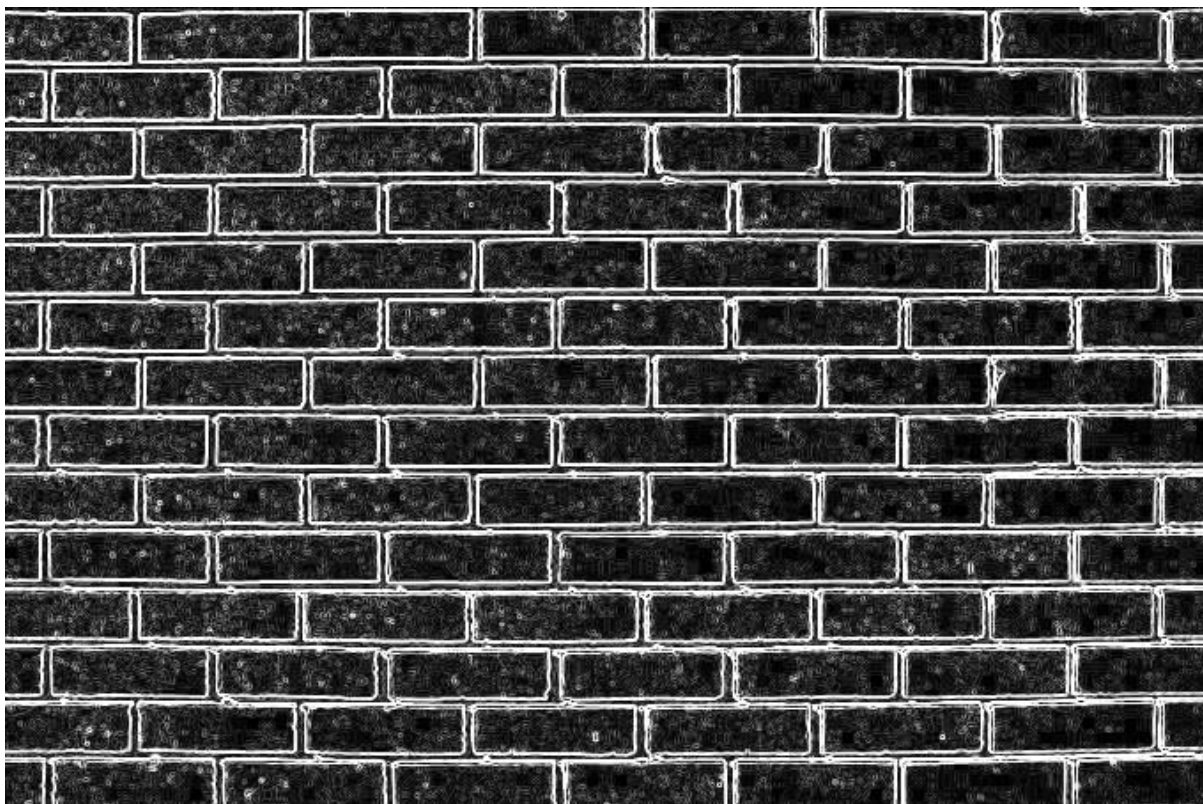


Figure 4: Gradient Magnitude

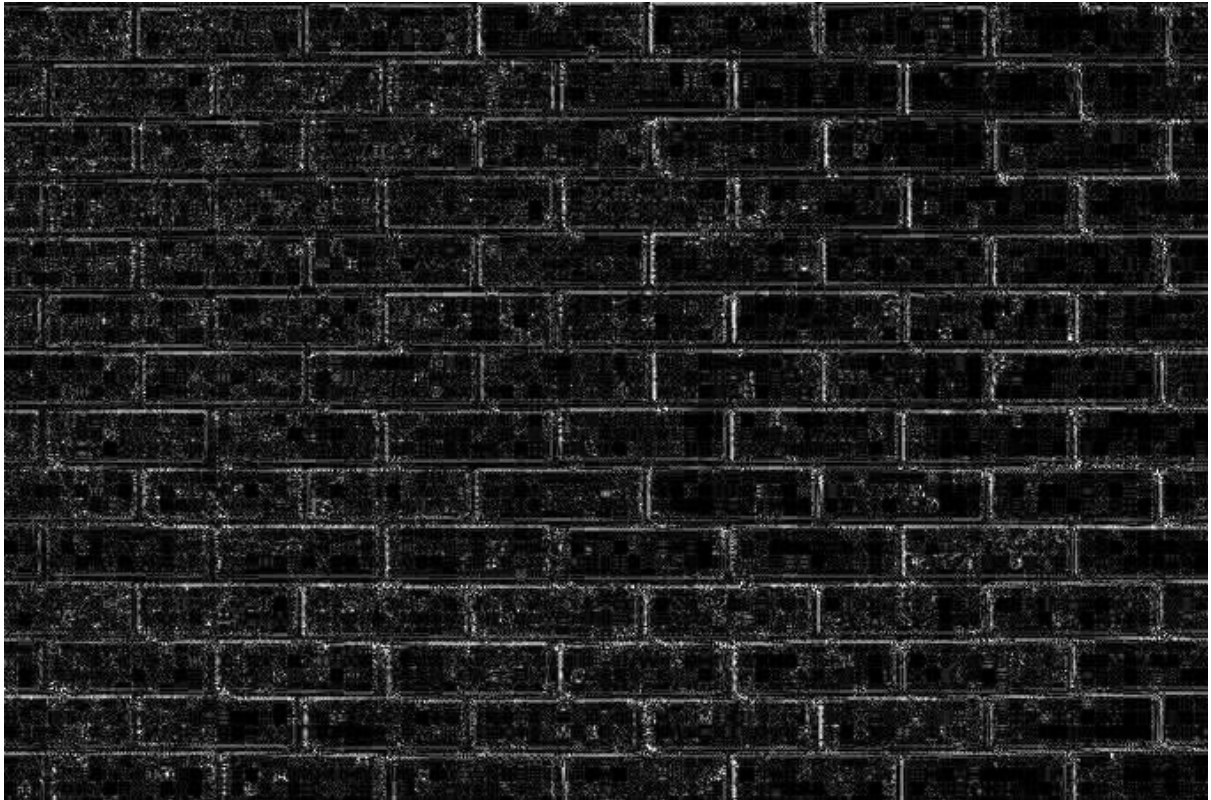


Figure 5: Laplacian Edge Detection

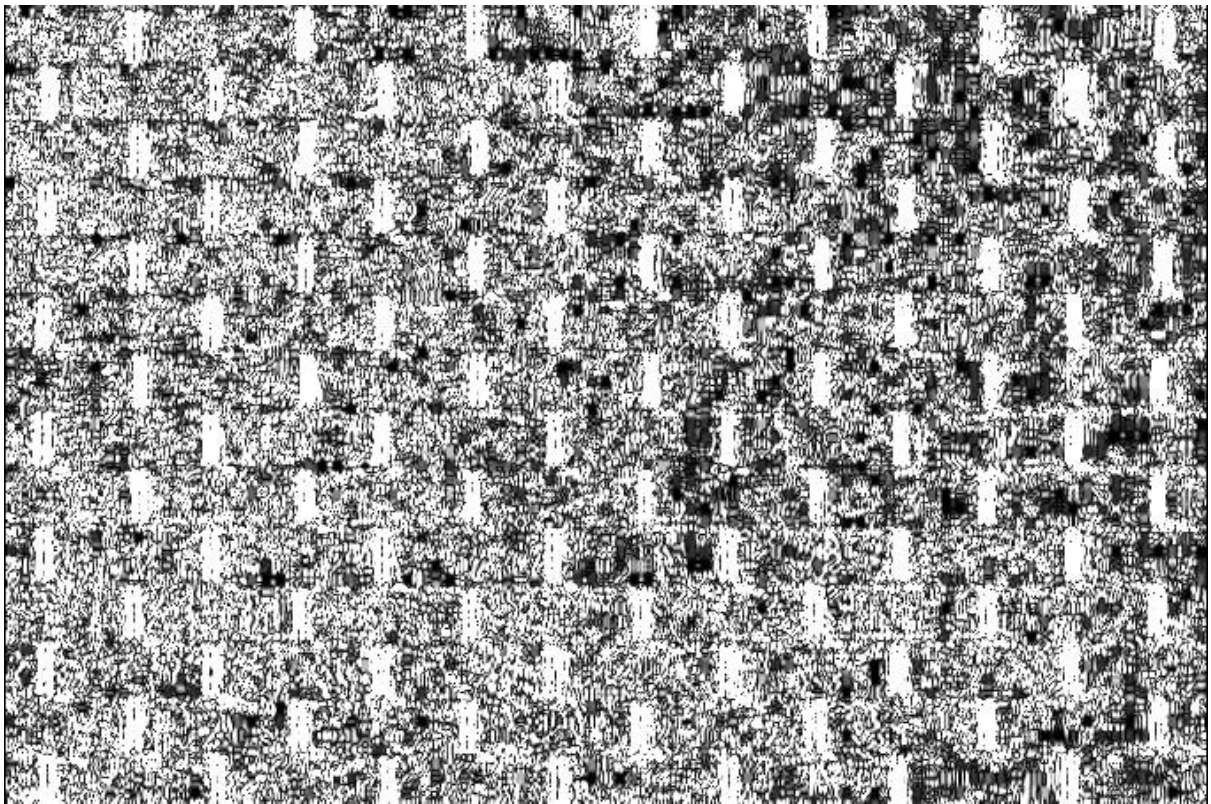


Figure 6: Sobel Edge Detection (5×5 Kernel)

Analysis

The Sobel operator detects edges by calculating the first-order derivative of image intensity. The Sobel X operator highlights vertical edges, while the Sobel Y operator highlights horizontal edges. Combining both responses produces the Gradient Magnitude, which provides a more complete representation of the image edges.

The Laplacian operator computes the second-order derivative and detects edges in all directions, making it useful for identifying fine details and rapid intensity changes.

Increasing the Sobel kernel size from 3×3 to 5×5 results in stronger and smoother edge responses because a larger neighbourhood is considered during filtering. However, very fine details may become slightly less distinct.

The experimental results show that each edge detection technique highlights different image features. Selecting an appropriate operator depends on the application and the desired level of edge enhancement.

Result

Edge enhancement was successfully performed using Sobel X, Sobel Y, Gradient Magnitude, and Laplacian operators. The comparison showed that the 5×5 Sobel kernel produces smoother and stronger edges, while the 3×3 kernel preserves finer edge details. The experiment demonstrated the effectiveness of different edge detection techniques in computer vision applications.